

This is the fifth of a series on object oriented programming.
In this presentation we will examine Polymorphism

Object Oriented Programming - Polymorphism

Polymorphism is a core concept in object-oriented programming that allows objects of different classes to be treated through the same interface. The word comes from Greek meaning 'many forms' - essentially, the same method name can behave differently depending on the object calling it.

Contents

- [Duck Typing](#)
- [Method Overriding](#)
- [Operator Overloading](#)
- [Function dispatch / Overloading alternatives \(Advanced\)](#)

Types of Polymorphism in Python

1. Duck Typing

Python uses 'duck typing' - if an object walks like a duck and quacks like a duck, it's treated as a duck. You don't need explicit inheritance; objects just need to have the required methods.

```
class Dog:
    def speak(self):
        return 'Woof!'

class Cat:
    def speak(self):
        return 'Meow!'

class Bird:
    def speak(self):
        return 'Chirp!'

def animal_sound(animal):
```

```

        print(animal.speak())

# All work despite no common parent class
animal = [Dog(), Cat(), Bird()]
for a in animal:
    animal_sound(a)

```

2. Method Overriding

Subclasses can override methods from their parent class to provide specific behavior. python

```

class Shape:
    def area(self):
        pass

class Circle(Shape):
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return 3.14 * self.radius ** 2

class Rectangle(Shape):
    def __init__(self, width, height):
        self.width = width
        self.height = height

    def area(self):
        return self.width * self.height

shapes = [Circle(5), Rectangle(4, 6)]
for shape in shapes:
    print(shape.area()) # Different calculation for each

```

3. Operator Overloading

You can define how operators work with your custom objects using special methods. python

```

class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

    def __str__(self):
        return f'Vector({self.x}, {self.y})'

v1 = Vector(2, 3)
v2 = Vector(5, 7)
v3 = v1 + v2 # Uses __add__
print(v3)    # Uses __str__

```

4. Function dispatch / Overloading alternatives (Advanced)

Python doesn't have compile-time overloading, but you can achieve behaviour variation with:

- type checks / default arguments
- `functools.singledispatch` (runtime dispatch based on the first argument's type)
- structural typing (`typing.Protocol`) for type hints

Example with *singledispatch*:

```
from functools import singledispatch

@singledispatch
def stringify(obj):
    return str(obj)

@stringify.register
def _(n: int):
    return f'int:{n}'

@stringify.register
def _(lst: list):
    return 'list:' + ','.join(map(str, lst))

print(stringify(5))          # -> int:5
print(stringify([1,2]))     # -> list:1,2
```

Helpful extras & advanced notes

- `__str__` vs `__repr__`: `__repr__` is for unambiguous representation (debugging); `__str__` is for a user-friendly string. Implement `__repr__`, and `__str__` can fallback to it.
- Use `typing.Protocol` to document required methods without inheritance (structural typing). Useful when you want type checking help but still rely on duck typing at runtime.
- Prefer duck typing for flexibility, but if many different types must implement exactly the same interface, consider a base class or an ABC to make intent explicit.

Benefits

- **Flexibility**: Write code works with many types
- **Extensibility**: Add new classes without changing consumers
- **Cleaner, more readable code**: Same interface for different implementations
- **Maintainability**: Easier to update and test

Polymorphism makes Python code more elegant and reusable, allowing you to write functions that work with any object that implements the expected interface. RetryClaude does not have the ability to run the code it generates yet. Claude can make mistakes. Please double-check responses.

Pitfalls

- Excessive duck typing can lead to surprising runtime `AttributeErrors` — consider tests or type hints.
- Overusing operator overloading may reduce readability if operators do non-obvious things.
- Incorrectly returning `NotImplemented` or failing to handle other types can break operations.

This is the last of a series on object oriented programming.

In this session we will examine **Abstraction**, one of the four foundational pillars of OOP- alongside **Encapsulation**, **Inheritance**, and **Polymorphism**.

Object Oriented Programming - Abstraction

What is Abstraction?

Abstraction means showing only the essential information to the user and hiding the complex internal details or implementation.

- Think of it like a remote control: When you press the 'Volume Up' button, the volume increases. You don't need to know how the circuit board, infrared signal, and TV processor work internally-you only interact with the simple interface (the button). The complexity is hidden.
- Goal: To simplify the view of a complex system. It deals with what an object does rather than how it does it.

Why Use Abstract?

- **Simplicity**: Users interact with clean, intuitive interfaces.
- **Reduced Complexity**: Focus on what an object does, not how it does it.
- **Maintainability**: Internal changes don't affect external usage.
- **Security**: sensitive logic remains hidden.
- **Structure Enforcement**: Ensures subclasses follow a consistent design.
- **Polymorphism Support**: Enables treating different objects as the same type (e.g., all animals can `make_sound()`).

How is Abstraction Achieved in Python?

Python achieves abstraction primarily through two mechanisms:

1. Creating Abstract Classes and Methods

- An Abstract Class is a class that cannot be instantiated (you can't create an object directly from it).
 - It's designed to be a blueprint or template for other classes.
 - May contain Abstract methods
 - May also contain concrete methods
- An Abstract Method is a method that has a declaration (a name, parameters) but no definition (no actual code/body). Subclasses must provide the definition (the implementation) for these methods.
 - Python's abc module: Python uses the built-in abc (Abstract Base Classes) module to define abstract classes and methods.
 - Key tools:
 - ABC: A metaclass used to define an abstract class.
 - @abstractmethod: A decorator used to declare a method as abstract.

2. Regular Classes with Method Organisation

```
class CoffeeMachine:
    def __init__(self):
        self._water_temperature = 20  # Internal state, marked with _
        self._beans = 10

    # Public Method: The simple interface for the user.
    def make_espresso(self):
        '''User just calls this to get coffee.'''
        if self._check_water() and self._check_beans():
            self._heat_water()
            self._grind_beans()
            self._brew()
            print('Here is your espresso! ☕')
        else:
            print('Cannot make coffee. Check water or beans.')

    # 'Private' Methods: The complex internals, marked with _.
    # The user SHOULD NOT call these directly.
    def _heat_water(self):
        print('Heating water to 96°C...')
        self._water_temperature = 96

    def _grind_beans(self):
        print('Grinding beans...')

    def _brew(self):
        print('Brewing espresso...')

    def _check_water(self):
```

```

        return self._water_temperature > 15 # Simplified check

    def _check_beans(self):
        return self._beans > 0

# Using the class
my_machine = CoffeeMachine()
my_machine.make_espresso() # This is correct and expected.

```

```

# The user CAN technically do this, but they SHOULDN'T.
# my_machine._heat_water() # This breaks the abstraction!
Heating water to 96°C...
Grinding beans...
Brewing espresso...
Here is your espresso! ☕

```

Note: Note: The single underscore `_` is a convention signaling that a method is intended for internal use. It's a 'gentleman's agreement' among developers-not enforced by Python, but respected in practice.

```

from abc import ABC, abstractmethod

```

```

# Abstract Class
class Animal(ABC):

```

```

    @abstractmethod
    def sound(self):
        pass

```

```

    @abstractmethod
    def move(self):
        pass

```

```

# Concrete Classes

```

```

class Dog(Animal):
    def sound(self):
        return 'Woof! Woof!'

    def move(self):
        return 'Running on four legs'

```

```

class Bird(Animal):
    def sound(self):
        return 'Chirp! Chirp!'

    def move(self):
        return 'Flying in the sky'

```

```

# Usage
things = [Dog(), Bird()]
for thing in things:
    print(f'{thing.__class__.__name__}: {thing.sound()}, {thing.move()}')

```

```
# This will raise an error:  
# animal = Animal() # TypeError: Can't instantiate abstract class
```

Key Components:

- **Abstract Class:** A class that cannot be instantiated (you can't create objects from it)
 - May have both abstract and regular methods
- **Abstract Method:** A method declared but contains no implementation
 - Use @abstractmethod decorator to mark methods as abstract.
- **Concrete Class:** A child class that inherits from abstract class and implements all abstract methods

Summary

1. **Standardization:** It forces all subclasses to follow a specific contract or interface (e.g., every shape must have an area() method and a perimeter() method).
2. **Maintainability:** It separates the common interface (in the abstract class) from the specific implementations (in the subclasses), making code easier to manage and modify.
3. **Clarity:** It hides the complex formulas/logic from the main user, allowing them to focus on what an object does (calculate area) rather than how it does it (e.g., using πr^2 vs. $l \times w$).